

BC01 — Présentation

Socle de données : architecture, enchaînement et rôles

Ce document explique simplement le premier bloc du projet : comment les données sont générées, stockées (PostgreSQL en étoile), ingérées en batch et en streaming (Kafka), et contrôlées avant d'alimenter BC02 → BC06.

Sommaire

1. Objectif du BC01
2. Architecture globale (schéma)
3. Les 4 couches PostgreSQL
4. Enchaînement batch vs streaming
5. Rôle de chaque script / fonction
6. Garanties (idempotence, qualité)
7. Comment démarrer / démontrer

1. Objectif du BC01

Poser les fondations data du projet anti-fraude : un entrepôt PostgreSQL en schéma étoile, un feature store Redis pour le temps réel, et un pipeline Kafka. Les blocs suivants (analyse, ML, API) consomment ce socle.

Choix clé : données synthétiques

seed.py génère 5 000 clients et 400 000 transactions (taux fraude ~1,5 %). Contrairement aux CSV Kaggle (variables PCA anonymes), on contrôle le schéma métier, les typologies de fraude et la volumétrie.

Livrables principaux :

- 4 schémas SQL : staging / dwh / mart / ops
- Une seule fonction de transformation batch + stream
- 10 contrôles qualité automatiques
- Infra Docker : Postgres, Redis, Kafka, Adminer, Kafka UI

2. Architecture globale

Deux voies d'ingestion (batch et stream) convergent vers la même porte SQL, puis vers le DWH et Redis.

BC01 — Architecture du socle de données

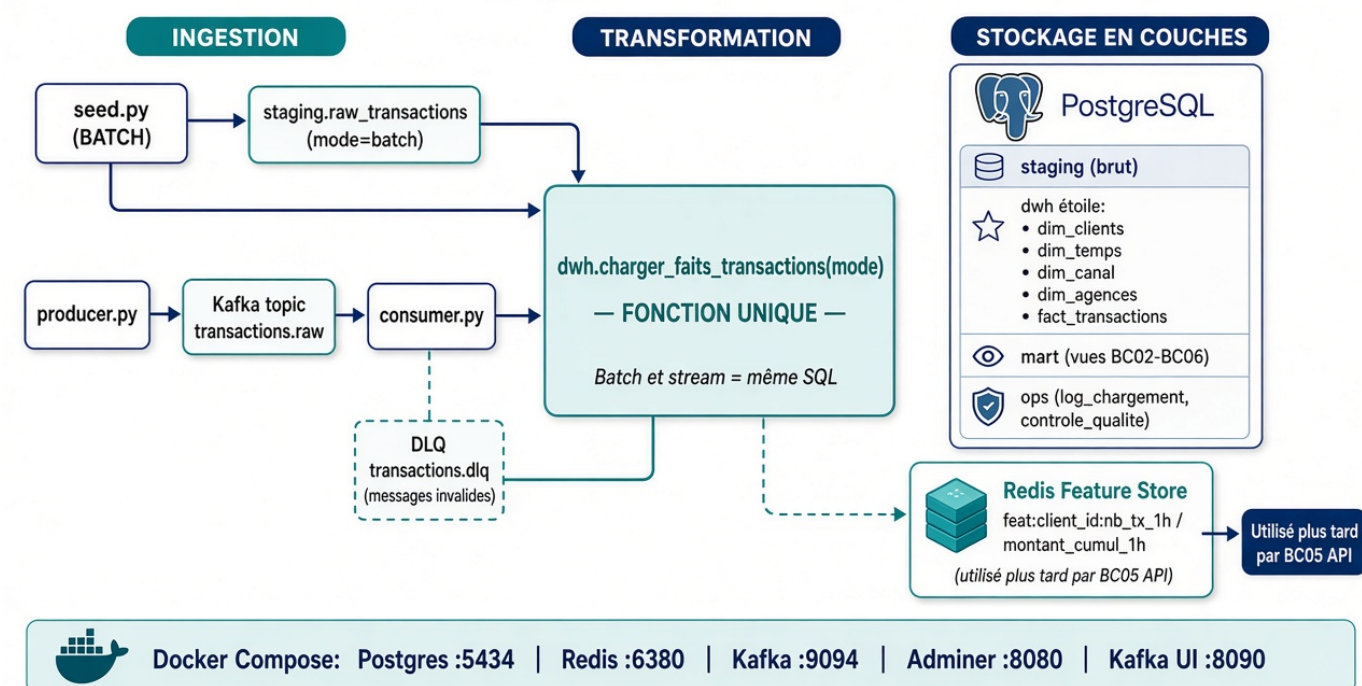


Figure 1 — Architecture BC01 (batch + streaming → DWH → mart / Redis)

Idée à retenir

Batch (`seed.py`) et streaming (`consumer.py`) appellent la MÊME fonction `dwh.charger_faits_transactions(mode)`. Les deux chemins ne peuvent pas diverger.

3. Les 4 couches PostgreSQL

Couche	Rôle	Exemples
staging	Copie brute, aucune règle métier	raw_transactions
dwh	Schéma en étoile (dimensions + faits)	dim_*, fact_transactions
mart	Vues pour BC02→BC06	v_kpis_globaux, v_profil_horaire
ops	Observabilité (hors chemin critique)	log_chargement, controle_qualite

Schéma en étoile (dwh)

Table	Rôle
dim_clients	5 000 clients (UUID, segment, revenu) — pas de nom (RGPD)
dim_agences	10 agences du réseau
dim_temps	1 ligne / heure + membre Inconnu (clé 0)
dim_canal	Canal × MCC + membre Inconnu (clé 0)
fact_transactions	1 ligne = 1 transaction carte
fact_card_blocks	Historique des blocages carte

Membre « Inconnu » (clé 0)

Si une transaction stream a un horodatage hors fenêtre 2025, elle est quand même écrite avec temps_key=0. Pas de perte silencieuse. Le client reste en jointure stricte (client inconnu = anomalie).

4. Enchaînement batch vs streaming

4.1 Voie batch (historique labellisé)

- 1. docker compose up -d (infra)
- 2. seed.py charge dimensions + staging (is_fraud déjà connu)
- 3. seed.py appelle charger_faits_transactions('batch')
- 4. quality_checks.py → 10/10
- 5. BC02→BC06 consomment fact_transactions (mode batch)

4.2 Voie streaming (temps réel)

- 1. consumer.py écoute Kafka (micro-lots 200 msgs / 2 s)
- 2. producer.py publie ~N txs/s sur transactions.raw
- 3. Validation JSON → DLQ si invalide
- 4. Dédup Redis → insert staging → charger_faits('stream')
- 5. Mise à jour Redis (nb_tx_1h, montant_cumul_1h) → API BC05
- 6. Commit offset Kafka APRÈS succès Postgres (at-least-once)

5. Rôle de chaque script / fonction

5.1 SQL (initialisation Docker)

Fichier	Rôle
00_schemas.sql	Crée staging, dwh, mart, ops
01_dwh_star.sql	Tables dimensions + faits
02_staging.sql	Table raw_transactions
03_ops.sql	log_chargement + controle_qualite
04_transform.sql	Fonction charger_faits_transactions
05_mart.sql	Vues KPIs / profil horaire / top fraude

5.2 data/seed.py (batch)

Fonction	Rôle
truncate_all	Vide les tables pour un rechargement propre
seed_unknown_members	Insère les membres Inconnu (clé 0)
seed_agences / seed_canal / seed_temps	Remplit les dimensions de référence
seed_clients	Génère 5 000 clients (UUID, segment, revenu)
seed_staging_transactions	Génère les txs labellisées + 6 typologies fraude
charger_faits	Appelle la fonction SQL unique (mode batch)
log_chargement_*	Journalise début/fin dans ops.log_chargement

5.3 streaming/producer.py

Fonction	Rôle
fetch_client_ids	Lit des client_id réels dans dim_clients
make_event	Construit un JSON transaction (clé métier unique)
main	Publie sur Kafka à un débit configurable

5.4 streaming/consumer.py

Fonction	Rôle
validate_event	Vérifie le JSON (sinon → DLQ)
is_duplicate / mark_seen	Déduplication Redis (TTL 24h)
flush_batch	Insert staging + charger_faits('stream')
update_online_features	Met à jour nb_tx_1h / montant_cumul_1h
send_to_dlq	Isole les messages malformés
log_chargement	Trace chaque micro-lot dans ops

5.5 Fonction SQL centrale

dwh.charger_faits_transactions(p_mode)

Lit staging (traite=FALSE, filtre batch/stream) → joint dim_clients (strict) + dim_temps/canal (LEFT + Inconnu) → insert fact_transactions ON CONFLICT DO NOTHING → marque traite=TRUE. Idempotente.

5.6 scripts/quality_checks.py

10 contrôles SQL journalisés dans ops.controle_qualite, par ex. :

- FK complètes dans fact_transactions
- Intégrité client_id staging ↔ dim_clients
- Unicité transaction_bk
- Cohérence montant / distance / label
- Taux de fraude plausible (0,5 % – 3 %)
- Fraîcheur staging (lignes non transformées)

6. Garanties

Propriété	Mécanisme
Idempotence batch	TRUNCATE + clés TX déterministes
Idempotence stream	ON CONFLICT DO NOTHING + dédup Redis
At-least-once	Commit Kafka après écriture Postgres
Pas de doublon producteur	enable.idempotence + acks=all
Messages invalides	Topic DLQ, flux non bloqué

7. Démarrer / démontrer

Commandes

```
docker compose up -d
python bc01_socle_donnees/data/seed.py --clients 5000 --transactions 400000 --fraud-rate 0.015
python bc01_socle_donnees/scripts/quality_checks.py
# optionnel streaming :
python bc01_socle_donnees/streaming/consumer.py
python bc01_socle_donnees/streaming/producer.py --rate 5
```

Interfaces utiles : Adminer <http://localhost:8080> · Kafka UI <http://localhost:8090>

Phrase soutenance (30 s)

« Le BC01 pose un DWH PostgreSQL en étoile avec quatre couches. Batch et streaming passent par la même fonction SQL. Kafka + Redis alimentent le temps réel, dix contrôles qualité valident le socle avant les blocs ML. »